

Connecting Codex CLI to Xcode's LLDB session

Participants: Claude & Shreeram Exported 24 August 2026

Source: <https://claude.ai/share/68fef39a-ad47-40e8-83c0-dfd179a07536>

This is a copy of a chat between Claude and Shreeram. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

SHREERAM 33 minutes ago

I want my codex cli to connect to lldb session started and owned by xcode.

I want my codex cli to connect to lldb session started and owned by xcode. I know there is an mcp for adding breakpoints and seeing stuff in the new xcode beta but it does not alert the codex when breakpoint hits. I want that to happen

CLAUDE 30 minutes ago

Short version: MCP has no path for waking a sleeping agent.

Searched the web

Searched the web

Short version: MCP has no path for waking a sleeping agent. Even if Xcode's server emitted a `notifications/*` message on a breakpoint hit, Codex would ignore it — the agent loop only acts during a tool call. So the fix isn't "make Xcode push"; it's **turn the event into a blocking tool call** the agent parks on.

Tier 0 — works today, zero code

Xcode 27's MCP `lldb` tool is synchronous against Xcode's live session. Issue `continue` through it with a long timeout and the call returns when the process stops. That is already a breakpoint-hit alert, just badly named. The default 30s timeout is what makes it useless; raise both the tool's own timeout param and Codex's:

TOML

```
[mcp_servers.xcode]
tool_timeout_sec = 600
```

Downsides: you only get whatever `continue` prints, no structured frame/locals, and it can't tell you which breakpoint without a follow-up round-trip.

Tier 1 — a notifier sidecar

Split responsibilities: Apple's MCP keeps doing all the commanding (po, bt, breakpoint set, continue), and you add a tiny observe-only bridge that does nothing but publish stop events. Two writers on one `SBDebugger` is where these setups wedge Xcode, so don't build a second command channel unless you have to.

Part 1: inside Xcode's LLDB. Xcode loads `~/lldbinit-Xcode` (it takes precedence over `~/lldbinit`, so if you have one, append there):

```
command script import ~/.lldb/xbridge.py
```

PYTHON

```
# ~/.lldb/xbridge.py - runs in Xcode's embedded Python. STDLIB ONLY, no pip.
import json, os, socket, threading, time, lldb

SOCK = os.path.expanduser("~/lldb/xbridge.sock")
_lock = threading.Lock()
_subs = []          # list of (conn, since_seq)
_hist = []          # recent events
_seq = 0

def _var(v):
    return {"name": v.GetName(), "type": v.GetTypeName(),
            "value": v.GetValue() or v.GetSummary()}

def _snapshot(frame):
    th = frame.GetThread(); le = frame.GetLineEntry()
    return {
        "pid": th.GetProcess().GetProcessID(),
        "ts": time.time(),
        "stop_reason": th.GetStopDescription(256),
        "thread": {"id": th.GetThreadID(), "queue": th.GetQueueName()},
        "file": le.GetFileSpec().fullpath if le.IsValid() else None,
        "line": le.GetLine() if le.IsValid() else None,
        "function": frame.GetFunctionName(),
        "args": [_var(v) for v in frame.GetVariables(True, False, False, True)],
        "locals": [_var(v) for v in frame.GetVariables(False, True, False, True)][0:40],
        "backtrace": [f"#{i} {f.GetFunctionName()}" for i, f in enumerate(th)][0:25],
    }

def _publish(ev):
    global _seq
    with _lock:
        _seq += 1
        ev["seq"] = _seq
        _hist.append(ev); del _hist[-50]
        line = (json.dumps(ev) + "\n").encode()
        for conn, _ in list(_subs):
            try: conn.sendall(line)
            except OSError: pass

class StopHook:
    def __init__(self, target, extra_args, internal_dict): pass
    def handle_stop(self, exe_ctx, stream):
        th = exe_ctx.GetThread()
        if th.GetStopReason() == lldb.eStopReasonBreakpoint:
            try: _publish(_snapshot(exe_ctx.GetFrame()))
            except Exception as e: stream.Print("xbridge: %s\n" % e)
        return True      # stay stopped

def _serve():
    try: os.unlink(SOCK)
    except OSError: pass
    srv = socket.socket(socket.AF_UNIX); srv.bind(SOCK); srv.listen(8)
    while True:
        conn, _ = srv.accept()
        try:
            since = json.loads(conn.recv(4096) or b"{}").get("since", 0)
            with _lock:
                backlog = [e for e in _hist if e["seq"] > since]
                _subs.append((conn, since))
            for e in backlog:
                conn.sendall((json.dumps(e) + "\n").encode())
```

```

        except Exception:
            conn.close()

def _install(debugger):
    ci = debugger.GetCommandInterpreter()
    seen = set()
    while True:
        try:
            for i in range(debugger.GetNumTargets()):
                t = debugger.GetTargetAtIndex(i)
                key = str(t.GetProcess().GetProcessID())
                if t.GetProcess().IsValid() and key not in seen:
                    r = lldb.SBCommandReturnObject()
                    ci.HandleCommand("target stop-hook add -P xbridge.StopHook", r)
                    seen.add(key)
        except Exception:
            pass
        time.sleep(0.5)

def __lldb_init_module(debugger, internal_dict):
    threading.Thread(target=_serve, daemon=True).start()
    threading.Thread(target=_install, args=(debugger,), daemon=True).start()

```

Stop hooks (rather than `SetScriptCallbackFunction`) matter here — breakpoint callbacks share the same slot as Xcode's UI breakpoint actions and will silently clobber them.

Part 2: the MCP server Codex talks to. The `since` seq is what makes this correct — without it the agent either misses hits that land between calls or replays stale ones.

PYTHON

```

#!/usr/bin/env python3
import json, os, socket
from mcp.server.fastmcp import FastMCP

SOCK = os.path.expanduser("~/lldb/xbridge.sock")
mcp = FastMCP("xcode-lldb-events")

@mcp.tool()
def wait_for_breakpoint(since_seq: int = 0, timeout_sec: float = 240) -> str:
    """Block until Xcode's debugger stops at a breakpoint. Returns the stop
    context (file, line, function, args, locals, backtrace) with a 'seq'.
    Pass that seq back as since_seq on the next call."""
    try:
        s = socket.socket(socket.AF_UNIX); s.connect(SOCK)
    except OSError as e:
        return json.dumps({"error": f"bridge not running in Xcode: {e}"})
    s.settimeout(timeout_sec)
    s.sendall(json.dumps({"since": since_seq}).encode())
    buf = b""
    try:
        while True:
            chunk = s.recv(65536)
            if not chunk: return json.dumps({"error": "bridge closed"})
            buf += chunk
            if b"\n" in buf:
                return buf.split(b"\n", 1)[0].decode()
    except socket.timeout:
        return json.dumps({"timeout": True, "since_seq": since_seq})
    finally:
        s.close()

if __name__ == "__main__":
    mcp.run()

```

TOML

```
# ~/.codex/config.toml
[mcp_servers.lldb_events]
command = "/opt/homebrew/bin/uv"
args = ["run", "--with", "mcp", "python", "/Users/shreeram/.lldb/xbridge_mcp.py"]
startup_timeout_sec = 30
tool_timeout_sec = 600      # MUST exceed timeout_sec above or Codex kills the call
```

Things that will bite you

Xcode's Python is not your Python. It's the interpreter inside `LLDB.framework` — no pip, no site-packages. Everything in `xbridge.py` is stdlib for that reason. During iteration, `script import importlib; importlib.reload(xbridge)` in the Xcode console, since `command script import` caches.

Never block in `handle_stop`. It runs on LLDB's internal thread; anything slow there freezes Xcode's debug bar. Snapshot and hand off, which is what `_publish` does.

The agent must be in the wait to hear anything. Put the loop in `AGENTS.md` explicitly, or Codex will call `wait_for_breakpoint` once and then wander off:

Debug loop: set breakpoints via the Xcode MCP → trigger the code path → call `wait_for_breakpoint` → inspect with the Xcode `lldb` tool using the returned frame → `continue` → call `wait_for_breakpoint` again with the previous `seq`.

`continue` and `wait` can't be in the same turn. The agent has to issue `continue` first, then park on `wait_for_breakpoint` — which means a small race if the breakpoint is hit instantly. The `since_seq` backlog covers it; that's the whole reason for `_hist`.

Stop hooks also fire on steps and crashes; I filtered to `eStopReasonBreakpoint`, but dropping the filter and passing `stop_reason` through is often more useful — a crash mid-run is exactly the thing you want Codex to notice.

SHREERAM 22 minutes ago

From codex

Claude's statement that there is "no path for waking a sleeping agent" is slightly too absolute for your installed `codex-cli` 0.149.1. Your CLI also has:

```
codex queue --thread <session-id-or-name> --message "Xcode breakpoint hit..."
```

Xcode officially supports running shell scripts as breakpoint actions, so a breakpoint action could call `codex queue` and inject a message into an existing Codex session. Apple documents breakpoint shell-script actions here (<https://developer.apple.com/documentation/xcode/setting-breakpoints-to-pause-your-running-app>). However, Codex's official documentation does not currently explain the wake/turn semantics of `codex queue`, so I would treat that as an experimental push-style option.

CLAUDE 21 minutes ago

Codex is right and I was overbroad.

Codex is right and I was overbroad. My claim holds for MCP itself — a server can't wake a client mid-idle through the protocol — but I then generalized it to "the agent loop only acts during a tool call," and `codex queue` is exactly a counterexample. It landed in 0.149.0 on 20 August, four days ago, and that release specifically notes queued messages now wake idle sessions reliably. So: genuine push, and it makes the blocking-tool design optional rather than mandatory.

ChatGPT Learn

Worth being precise about the semantics, though, because it changes which design you want. The command delivers a message to an existing local or remote session without interrupting the active turn. Two cases: [Codex Knowledge Base](#)

- **Session idle** → the message wakes it and starts a turn. True push.
- **Session mid-turn** → it queues and lands at the turn boundary. No preemption.

For most async events that second case would be lossy. Here it isn't, and that's the thing that makes this work well: **a breakpoint hit isn't a notification, it's persistent state**. The process is frozen and stays frozen. Whether Codex reads the message in 200ms or 90 seconds, the stop is still there waiting. Latency costs you nothing.

Which design, and when

The dividing line is who's driving the app.

Agent drives (it set the breakpoint, it triggered the code path) → blocking `wait_for_breakpoint`. It's already mid-turn and stays there; one round trip.

You drive (you're tapping through the UI, or QA is, or it's a heisenbug you're reproducing by hand) → `codex queue`. The agent sits idle, you exercise the app, the breakpoint wakes it. This is the case my design handled badly and Codex's handles well.

Don't run both. A queued message can't be delivered while `wait_for_breakpoint` is blocking, so combining them just adds a deadlock window.

Wiring it

Since you already have the stop hook, don't bother with per-breakpoint shell actions — have the hook spawn the notify itself, and every breakpoint gets it for free:

PYTHON

```
import subprocess
SNAP = os.path.expanduser("~/lldb/last-stop.json")
CODEX = "/opt/homebrew/bin/codex" # absolute: breakpoint context has a thin PATH

def _publish(ev):
    with open(SNAP, "w") as f:
        json.dump(ev, f)
    msg = (f"Xcode breakpoint hit: {ev['function']} at "
          f"{os.path.basename(ev['file'] or '?')}:{ev['line']}. "
          f"Full stop context (args, locals, backtrace) is in {SNAP}. "
          f"The process is stopped and waiting. Inspect with the Xcode lldb "
          f"tool, then continue when done.")
    subprocess.Popen([CODEX, "queue", "--thread", "xcode-debug", "--message", msg],
                     stdin=subprocess.DEVNULL, stdout=subprocess.DEVNULL,
                     stderr=subprocess.DEVNULL, start_new_session=True)
```

`start_new_session=True` and the devnull redirects are load-bearing — this runs on LLDB's internal thread, and anything that blocks there hangs Xcode's debug bar before the stop is even published. Same reason the Apple shell-action route needs **"Wait until done" unchecked** in the breakpoint editor (and "Automatically continue" left checked off, or the process resumes before Codex reads anything).

The queued message carries only a pointer, not the payload. Deliberate — the snapshot goes to a file, the agent reads it if it needs locals, and you don't burn a few thousand tokens of backtrace on every hit.

`--thread "xcode-debug"` requires a named session, and Codex rejects the name as ambiguous if several sessions share it, so name it once and don't run two. [Hammerautomation](https://hammerautomation.ai/en/forge/openai-codex-release-notes-2026-08-21)

The one thing I'd still verify yourself

Codex's hedge about undocumented turn semantics is the right instinct, and there's a specific reason: "wake idle sessions reliably" reads as a fix shipped in that same release, which means it was flaky before it. On 0.149.1 you're past it, but I'd test the idle-wake path deliberately — start the session, leave it untouched for a few minutes so it's genuinely dormant rather than freshly idle, then trigger a breakpoint — before you build a workflow that assumes it.

Also worth checking: whether a woken turn inherits the approval mode you expect. If it wakes and immediately needs you to approve an `lldb` tool call, you've replaced one manual step with another.

Links referenced

1. Apple — Setting breakpoints to pause your running app
<https://developer.apple.com/documentation/xcode/setting-breakpoints-to-pause-your-running-app>
2. Codex changelog
<https://developers.openai.com/codex/changelog>
3. Codex Knowledge Base — codex queue inter-session messaging (v0.149)
<https://codex.danielvaughan.com/2026/08/21/codex-queue-inter-session-messaging-codex-cli-v0149-orchestration-automation-agent-to-agent/>
4. Hammerautomation — OpenAI Codex release notes 2026-08-21
<https://hammerautomation.ai/en/forge/openai-codex-release-notes-2026-08-21>